minicut Video Editor

Embedded JavaScript & Plugin API Reference Manual

Duktape 2.7.0 Engine Full UTF-8 / Multi-lingual Dockable Tool Panels Tool Shelf Dock Network

Library (libcurl)

1. Introduction & JavaScript Architecture

minicut embeds a complete **JavaScript** engine (Duktape 2.7.0) with native UTF-8 and Unicode string support. JavaScript plugins provide complete parity with the minicut scripting API, making it easy to create internationalized tools across all supported languages.

Script Directory Locations

Location	Trigger / Role	Description
`~/.config/minicut/autorun/`	Startup / Auto-register	`.js` files execute on startup and generate an icon button on the horizontal Tool Shelf Dock below the main menu bar.
`~/.config/minicut/scripts/`	Menu Click / On Demand	`.js` files appear directly in the top-level **Modifiers** menu on the main menu bar.
`~/.config/minicut/shaders/gl/`	Custom GL Shaders	User OpenGL vertex (`.vert`) and fragment (`.frag`) shaders loaded by name via `shader.create("name")`.

Location	Trigger / Role	Description
`~/.config/minicut/shaders/vulkan/`	Custom Vulkan Shaders	User Vulkan GLSL vertex and fragment shaders compiled for the Vulkan rendering backend.
`~/.config/minicut/icons/`	Dock Icon Asset	PNG icon images matching script stems (e.g., `quick_fade_js.png` for `quick_fade_js.js`).

2. JavaScript Plugin Architecture

```
// ================================================================================
// Example JavaScript Plugin: sample_tool.js
// ================================================================================

var targetTrack = 0;
var clipDuration = 4.0;
var mediaPath = "tests/assets/test_sample.png";

function placeMedia() {
    minicut.push_undo();
    var playhead = timeline.get_playhead();
    var cid = timeline.place_clip(mediaPath, targetTrack, playhead, clipDuration);
    if (cid && cid.length > 0) {
        clip.set_scale(cid, 1.25);
        timeline.add_keyframe(cid, "opacity", 0.0, 0.0);
        timeline.add_keyframe(cid, "opacity", 1.0, 1.0);
        minicut.set_status("Placed media clip: " + cid);
```

```
    }
}


function browseFile() {
    gui.file_dialog("Select Media", function(selected) {
        if (selected && selected.length > 0) {
            mediaPath = selected;
            minicut.set_status("Selected: " + selected);
        }
    });
}


plugin.init_panel = function() {
    gui.text_colored(0.24, 0.68, 0.91, 1.0, "Media Placer (JS)");
    gui.separator();
    targetTrack = gui.slider_int("Track Lane", targetTrack, 0, 4);
    clipDuration = gui.slider_float("Duration", clipDuration, 0.5, 30.0);
    gui.button("Browse Media...", browseFile);
    gui.button("Place on Timeline", placeMedia);
};
```

3. JavaScript Modules API

3.1. Project & File Automation (`project` / Global)

Function	Returns	Description
`project_save([path])` `project.save([path])`	`boolean`	Saves project to `.minicut` JSON file.
`project_load(path)` `project.load(path)`	`boolean`	Loads project from path.
`project.get_name()`	`string`	Returns current project name.
`project.set_name(name)`	`void`	Sets current project name.

3.2. Network & Internet Library (`net`)

Function	Returns	Description
`net.get(url)`	`object`	HTTP GET request returning `{status, body, ok, headers}`.
`net.post(url, body)`	`object`	HTTP POST request.
`net.download(url, dest)`	`boolean`	Downloads remote asset directly to file (supports HTTP, HTTPS, FTP, SFTP).
`net.upload(url, path)`	`object`	Uploads local file binary stream (supports HTTP multipart, FTP, and secure SFTP).

3.3. Timeline & Track Management (`timeline`)

Function	Returns	Description
`timeline.get_playhead()`	`number`	Gets playhead time in seconds.
`timeline.set_playhead(t)`	`void`	Sets playhead time in seconds.
`timeline.place_clip(path, track, t,`	`strin`	Places media clip on track and returns clip

Function	Returns	Description
`dur)`	`g`	ID.
`timeline.add_keyframe(id, prop, t, val)`	`void`	Adds keyframe for property at timestamp.
`timeline.clear_keyframes(id, prop)`	`void`	Clears all keyframes for property.
`timeline.select_clips(ids_array)`	`void`	Selects multiple clips simultaneously.
`timeline.move_track_up(idx)`	`void`	Moves track up in stack.
`timeline.move_track_down(idx)`	`void`	Moves track down in stack.
`timeline.set_track_name(idx, name)`	`void`	Renames track at index.

3.4. Clip Attributes & Transforms (`clip`)

Function	Returns	Description
`clip.get_position(id)`	`array [x, y]`	Gets position coordinates relative to canvas center.
`clip.set_position(id, x, y)`	`void`	Sets center position coordinates.
`clip.get_scale(id)`	`number`	Gets uniform scale factor (1.0 = 100%).
`clip.set_scale(id, s)`	`void`	Sets uniform scale factor.
`clip.get_opacity(id)`	`number`	Gets opacity [0.0, 1.0].
`clip.set_opacity(id, o)`	`void`	Sets opacity.
`clip.get_volume(id)`	`number`	Gets audio volume gain multiplier [0.0, 2.0].
`clip.set_volume(id, v)`	`void`	Sets audio volume gain multiplier.
`clip.get_transform(id)`	`object`	Gets transform object `{x, y, scale, rotation}`.
`clip.set_transform(id, x,`	`void`	Sets translation X, Y, scale, and rotation

Function	Returns	Description
`y, s, r)`		simultaneously.
`clip.reset_transform(id)`	`void`	Resets clip transform to origin (x=0, y=0, scale=1.0, rotation=0).
`clip.get_frame(id, [t, w, h])`	`object`	Extracts raw RGBA frame object. Pass `"active"` for selected clip. Supports fluent chaining (`.apply_shader().reinsert()`).
`clip.set_frame(id, frame, [t])`	`boolean`	Reinserts modified RGBA pixel frame into clip media. Triggers full-clip rendering if shader attached.

3.5. GUI Controls (`gui`)

Function	Returns	Description
`gui.text(text)`	`void`	Renders text label.
`gui.text_colored(r, g, b, a, text)`	`void`	Renders colored text with RGBA.
`gui.button(label, [callback])`	`boolean`	Renders button and triggers callback on click.
`gui.slider_float(label, val, min, max)`	`number`	Renders float slider.
`gui.slider_int(label, val, min, max)`	`integer`	Renders integer slider.
`gui.input_text(label, text)`	`string`	Renders text entry box.
`gui.file_dialog(title, cb, [filter])`	`void`	Opens file dialog matching user settings.
`gui.popout_preview(popped)`	`void`	Detaches (true) or docks (false) the video preview panel into a floating window.
`gui.is_preview_popped_out()`	`boolea`	Returns whether preview is currently

Function	Returns	Description
	`n`	detached/floating.
`gui.set_preview_aspect_mode(mode)`	`void`	Sets aspect mode: 0 (Auto), 1 (16:9), 2 (9:16), 3 (4:3), 4 (1:1), 5 (21:9), 6 (4:5).
`gui.get_preview_aspect_mode()`	`number`	Returns current preview aspect mode index (0-6).
`gui.set_transform_tool(tool)`	`void`	Sets active preview gizmo tool ("select", "translate", "scale", "rotate").
`gui.get_transform_tool()`	`string`	Returns active preview gizmo tool name.

3.6. Export Automation & Progress (`export`)

Function	Returns	Description
`export.get_audio_progress()`	`number`	Audio mixing progress (stereo mixing & keyframes, 0.0 to 1.0).
`export.get_progress()`	`number`	Overall export pipeline progress (0.0 to 1.0).
`export.is_exporting()`	`boolean`	True if background render/encode is actively running.
`export.get_status()`	`string`	Human-readable pipeline status description.

3.7. Custom GLSL Shader Pipeline (`shader`)

Loads custom shaders from `~/.config/minicut/shaders/gl/` and `~/.config/minicut/shaders/vulkan/` by name (e.g. `"ripple"`, `"grayscale"`, `"invert"`, `"vignette"`), or compiles custom GLSL fragment source strings. When applied to a clip, the shader processes the **entire clip frame-by-frame** in the background using FFmpeg without blocking UI responsiveness.

Function	Returns	Description
`shader.create(name_or_src)`	`number`	Loads shader by name from shaders directory or compiles in-memory GLSL source.
`shader.set_uniform(id, name, val)`	`void`	Sets float, int, or object vector uniform (matches with or without `u_` prefix).
`shader.get_uniform(id, name)`	`number / object`	Returns current uniform parameter value.
`shader.apply(id, target, [uniforms])`	`boolean / object`	Renders shader across all video frames of clip (or transforms single frame buffer).

Frame Buffer Fluent Piping

```
// Fluent method chaining across entire clip

clip.get_frame("active").apply_shader("ripple", { amplitude: 12.0, frequency:
0.08 }).reinsert();



// Or synthesize a brand-new clip on track 0

clip.get_frame("active").apply_shader("ripple").create_clip(0, 0.0, 5.0, "Ripple Wave");
```